CS 543 Web Services — Assignment 2 ReadMe

Submission Brief & Architecture Benchmark for BookMe

Platform: BookMe **Team:** 11 · Semester Submission **Date:** October 2026

👥 Team 11 Members & Service Allocations

Member Name	University Roll No.	Assigned Service	Service Codebase Directory
Radhika Verma	`20251651075`	Auth Service	`bookme/services/auth-service`
Nandini Kanaujiya	`20251651061`	Vendor Service	`bookme/services/vendor-service`
Alka Jha	`20251651013`	Event & Booking Service (Central)	`bookme/services/event-booking-service`
Alok Mishra	`20251651014`	Payment & Feedback Service	`bookme/services/payment-feedback-service`

🔗 Submission Link

- **GitHub Repository:** https://github.com/Nandinikanaujiya/BookMe_Assignment

📦 Submission Contents & Deliverables

Deliverable	File Location in Repo	Task & Description
Design Benchmark Document	`Assignment-2/design.pdf`	Capabilities (Task 1), Contracts (Task 3), `createBooking` Full Spec (Task 4), Validation Table (Task 6).
Service Design Benchmark	`Assignment-2/services.drawio` / `.png`	Architecture diagram mapping 4 microservices, data ownership, team members, and inter-service call operations (Task 2).
Database Schema ERD	`Assignment-2/schema.drawio` / `.png`	Entity-Relationship Diagram depicting service schemas, attributes, PK/FKs, and logical references (Task 5).
Database DDL (SQL)	`Assignment-2/schema.sql`	PostgreSQL table creation queries enforcing zero shared database tables (Task 5).
Technical Architecture Brief	`bookme/docs/assignment-2.md`	Comprehensive documentation mapping all 6 tasks directly to BookMe codebase modules.
Project Overview Brief	`README.md` / `ReadMe.pdf`	Submission brief containing complete explanations, links, and validation criteria.

📋 Summary of Implementation Across Tasks

- **Task 1 (Capabilities):** Identified 7 discrete system capabilities directly derived from `bookme/brief.md`: `registerAndAuthenticateUser` (Auth), `manageVendorCatalog` (Vendor), `manageCalendarAvailability` (Vendor), `planEventAndBudget` (Event-Booking), `createBooking` (Event-Booking), `processEscrowPayment` (Payment-Feedback), and `submitFeedbackAndDisputes` (Payment-Feedback).
- **Task 2 (Service Architecture):** Modeled 4 microservices with assigned leads and distinct data stores. Labeled all inter-service interaction calls (`verifyCustomer`, `checkAvailability`, `reserveSlot`, `initiatePayment`, `updateBookingStatus`, `getVendorPayoutAccount`).
- **Task 3 (Service Contracts):** Defined rigorous API contracts with inputs, outputs, and errors for all 4 services, maintaining information hiding (zero database/SQL exposure).
- **Task 4 (Central Operation):** Specified `createBooking` in full detail, including inputs, outputs, 6 HTTP error statuses, and internal encapsulation details (database atomicity, rate freezing snapshots, two-phase availability verification).
- **Task 5 (Database Schema):** Created private schemas for all 4 services in PostgreSQL DDL and visual draw.io ER diagram, completely avoiding cross-database joins and foreign keys.
- **Task 6 (Service Property Validation):** Validated all services against 5 core properties (Reachable, Self-contained, Has a contract, Independent, Loosely coupled), noting latency coupling trade-offs and remediation via caching and async sagas.